Logical Architecture and Implementation of FABLE

08/28/26

Preliminaries: Complex Event Types

- We ran experiments on a variety of different complex events

Robberies

Vehicle convoys

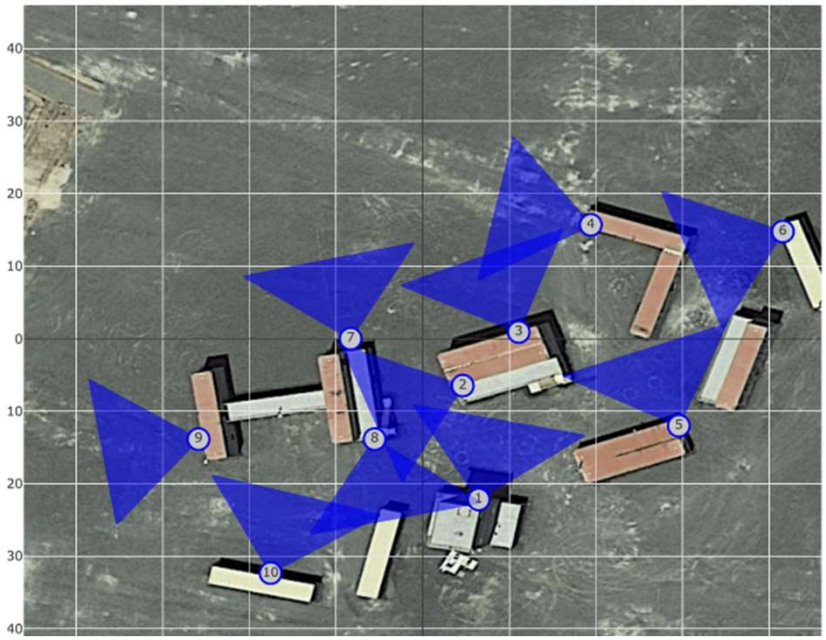
Vehicle chase

Stalking

Converging Operation

...

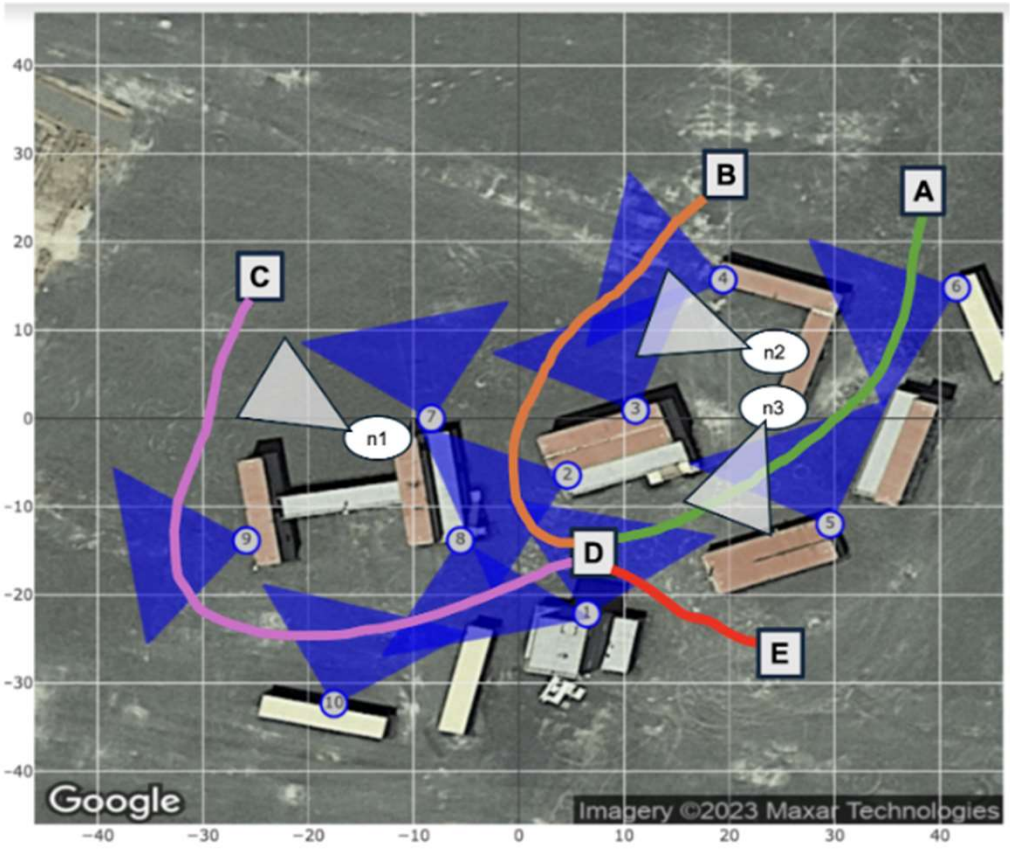
Preliminaries: Experimentation Site



Testing site

Blue triangles are sensing devices

Preliminaries: 2024 Example (Graces Quarters)

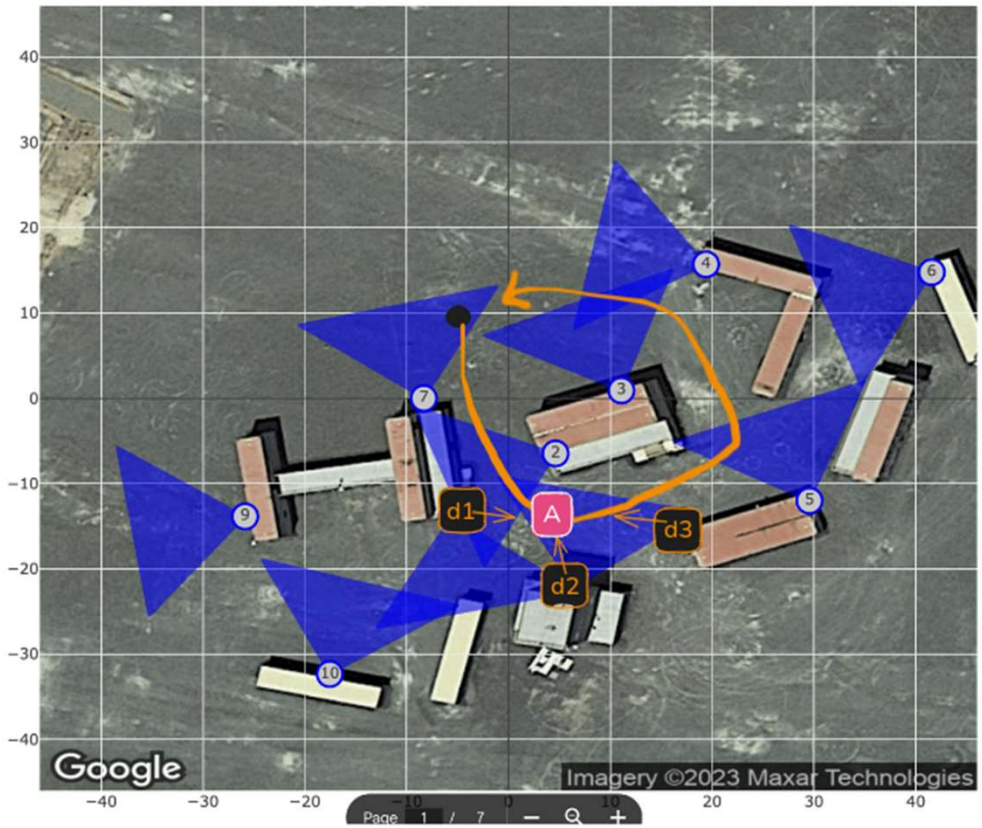


Complex Event
ID: spatial_ce1

Additional Nodes:
- n1, n2, n3 are our own mobile devices recording and running YOLO detection, and publishing to MQTT

Node Notes:
- These nodes are mobile devices which can be mounted with a magnetic bracket on the walls of a shipping container.
- WE WILL NEED POWER CORDS FOR THESE NODES
- We will start/stop their recording remotely at the beginning/end of our experiment block

Preliminaries: 2025 Example (Graces Quarters)



Shooting

Vehicle 1: Orange line

Black dots: starting points
d1-d3: our cameras
pink locations: POI
red dots: objects

Path:
- Vehicle 1 arrives at A, pauses
- Gunshot sounds at d1
- Vehicle 1 leaves
- Restart complex event

Preliminaries: 2026 Example (West Point)

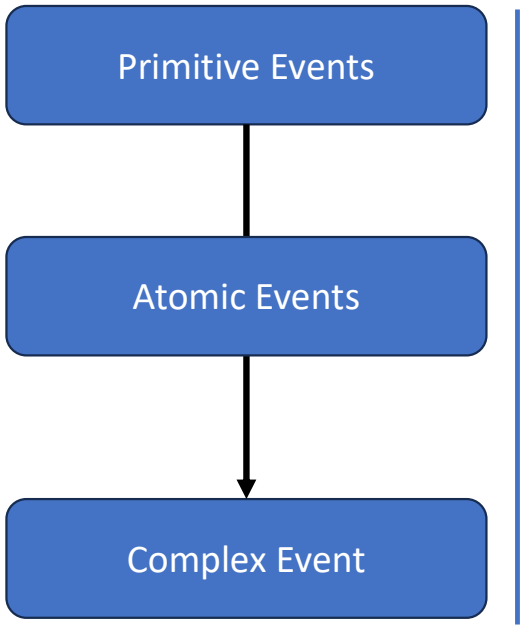
Note:

- Did not have mobile phone sensors
- IoBT nodes were mobile and changed across CEs

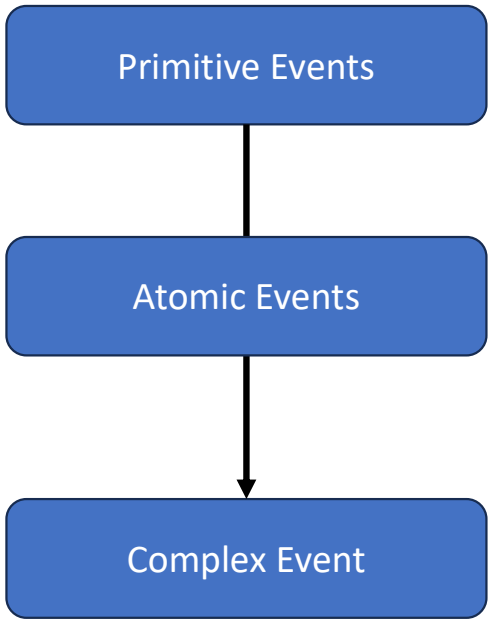
Recorded maps, CE experiments

- 2024 Cases:
  - https://drive.google.com/drive/folders/1IpAEn7DzZA-QwtjAsBnMlXtngYOB10xV?usp=sharing
- 2025 Cases:
  - https://drive.google.com/drive/folders/1uziuGQpD-ZAEfkMg1ndLlTJ70R3ddQEk?usp=drive_link
- 2026 Cases:
  - https://drive.google.com/drive/folders/1xcHnRIgg36u0KdLSbGlUdREtXr4tkKMu?usp=drive_link

Preliminaries: CE Hierachy

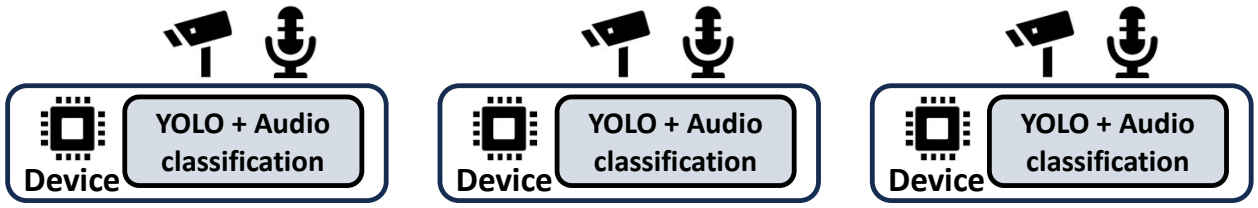


Preliminaries: CE Hierachy



	Description	**Space/time**	**Defined by**
Primitive Events	Derived from sensory data (e.g. YOLO detections)	Instant in time, single sensor	Decided by implementation
Atomic Events	Operate over primitive events (e.g. count number of red vehicles)	Instant in time, multiple sensor	Specified by user
Complex Event	Operate over atomic events	Long period of time, multiple sensors	Specified by user

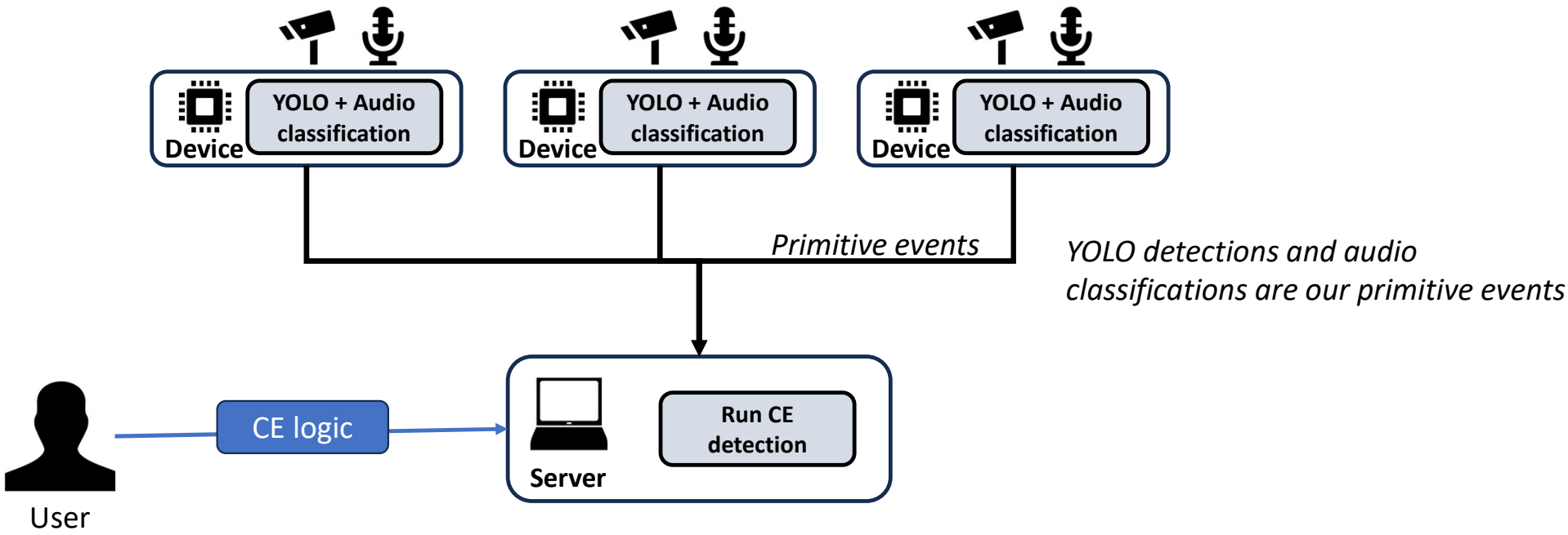
Preliminaries: CE detection (previous work)



Devices:

- Android devices (Pixel 8, Pixel 9, Pixel 9 pro) directly running YOLOv8s and YamNet
- IoBT orin nodes running YOLOv8s and YamNet as containers

Preliminaries: CE detection (previous work)



Preliminaries: Example CE logic

Atomic Events (combined into CE)

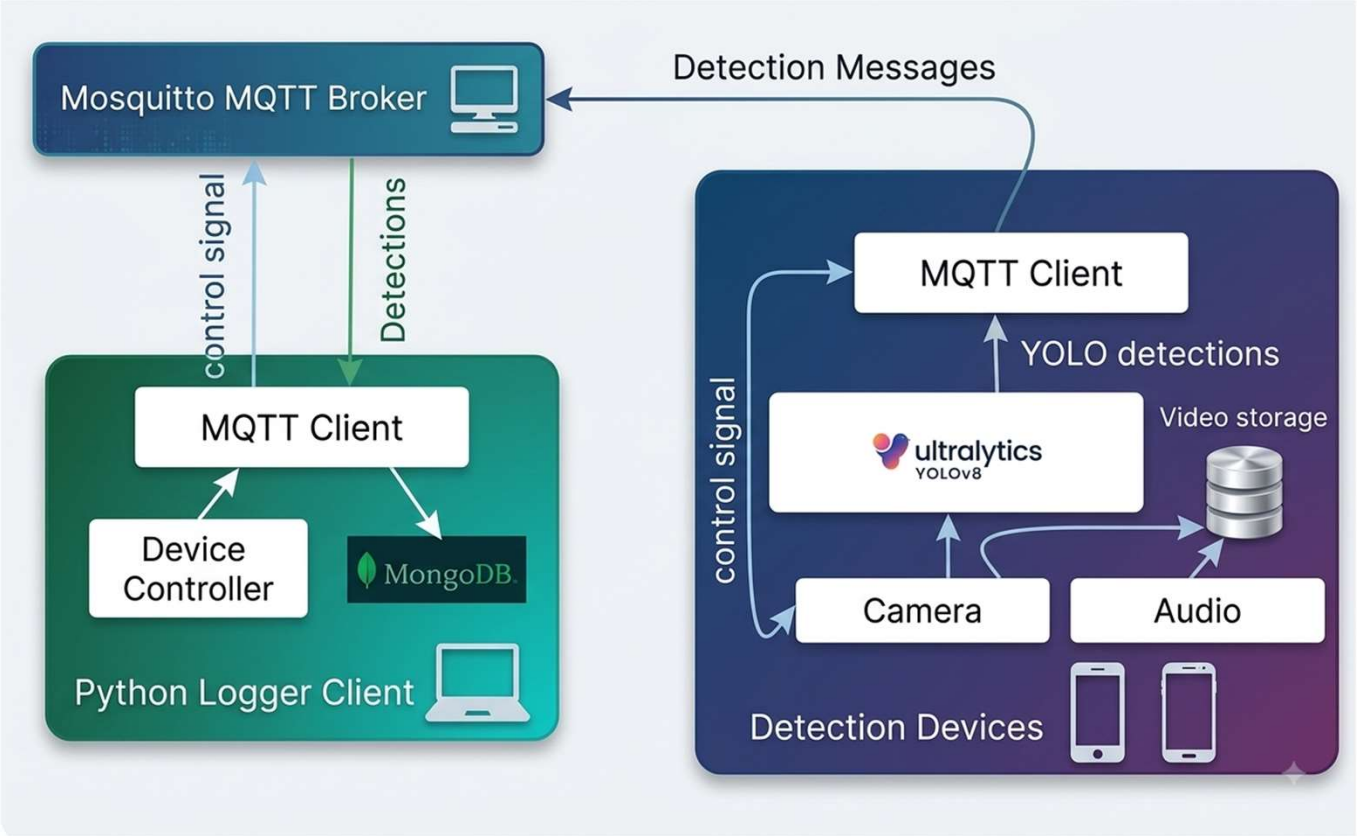
```
outer_node = "node14"
building_node = "node15"

# Create FSM states
# State 1: Initial sedan seen at outside camera node
state1 = FSMState("Sedan detected at " + outer_node)
state1.set_trigger_list([(outer_node, one_sedan, "class_name", None), (outer_node, color_of_interest, "color", None)])

# State 2: Person leaves vehicle at outside camera node
state2 = FSMState("Person outside building at " + outer_node)
state2.set_trigger_list([(outer_node, one_person, "class_name", None)])

# State 3: Person enters building, seen at building camera node
state3 = FSMState("Person enters building at " + building_node)
state3.set_trigger_list([(building_node, one_person, "class_name", None)])

# State 4: Gunshot heard in building
state4 = FSMState("Gunshot at building " + building_node)
state4.set_trigger_list([(building_node, gunshot_heard, "class_name", None)])

# State 5: Person leaves outside of building, seen at outside camera node
state5 = FSMState("Person leaves building, seen outside at " + outer_node)
state5.set_trigger_list([(outer_node, one_person, "class_name", None)])

# State 6: Person and vehicle are gone
state6 = FSMState("Person and vehicle gone " + outer_node)
state6.set_trigger_list([(outer_node, no_person_or_vehicle, "class_name", None)])
```

Functions used to determine atomic events

```
# Define the functions which trigger state transitions
def one_sedan(value):
    return (value.count("car")) >= 1

def one_person(value):
    return (value.count("person")) >= 1

def color_of_interest(value):
    return (value.count("red")) >= 1
```

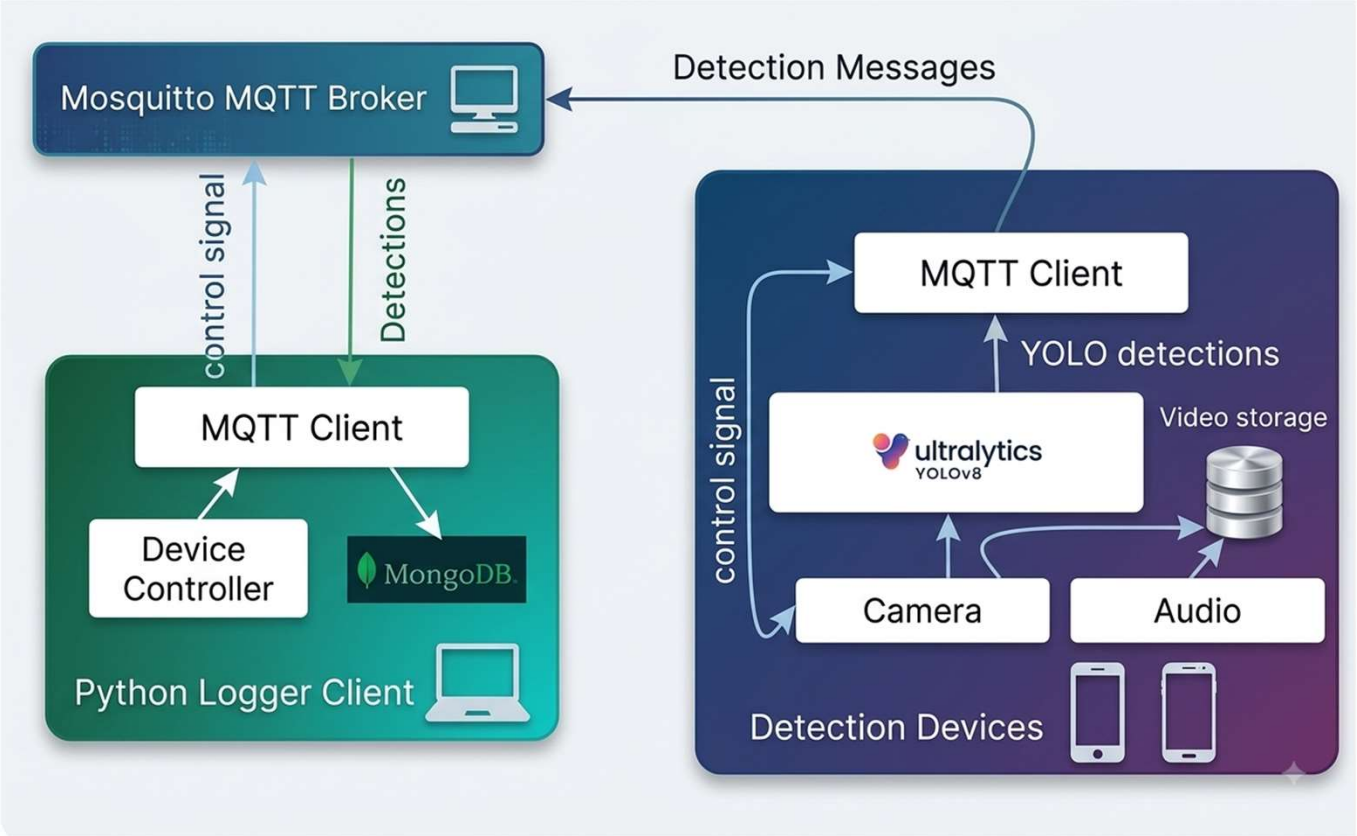
Note – deployment information (e.g. specific node) is part of logic

Early Architecture



*note – you can consider Yamnet to be part of the YOLO block

Early Architecture



*note – you can consider Yamnet to be part of the YOLO block

Primitive event detectors (e.g. YOLO, Yamnet) are always on.

All devices are treated the same (same detectors), regardless of compute/network

Does not yet include a way to retrieve historical information useful for CE detection

Limitations of previous architecture

- CE definition is not decoupled from deployment information
  - E.g. user has to specify they want to monitor YOLO detections from orin14
- Detection and classification are always on, leading to resource inefficiencies
- Different physical nodes may have different compute/network characteristics, requiring different choices of models
- No way to examine historical evidence at runtime

Questions:

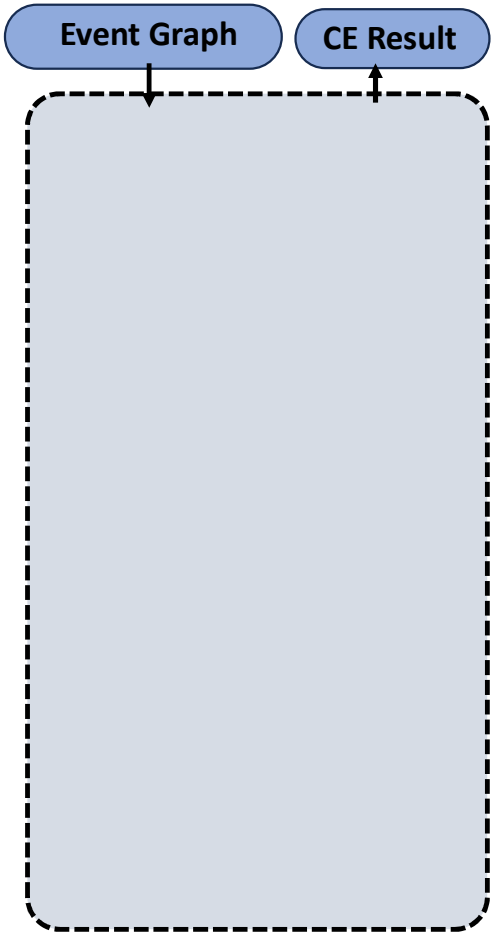
1. **What is the CE representation?**

2. **Given the CE representation, what do we need to monitor for?**

3. **What additional constraints are there for how monitoring is done?**

4. **What is the concrete implementation for performing that monitoring?**

5. **How is the concrete implementation distributed across devices?**



Questions:

1. **What is the CE representation?**

Complex event structure

Convoy / chase

1. Vehicle A passes
2. Vehicle B follows A
3. Vehicles clear area

Entities: vehicles

Behaviors: Vehicles passing, vehicles following, Vehicles departing

Structure: ordered progression

Robbery

1. Vehicle is present
2. Gunshot / alarm
3. Vehicle later departs

Entities: vehicle, person

Behaviors: Vehicles present, Gunshot detection, Vehicles departing

Structure: earlier / later temporal relationship

Package exchange

1. Person arrives with package
2. OR (direct handoff, drop / pickup)
3. Person leaves

Entities: person, package

Behaviors: Objects detected, Objects and people interactions

Structure: alternative manifestations + bounded timing

Talking / rendezvous

1. Two people near each other
2. Speech activity occurs
3. Continues for some time

Entities: people

Predicates: People near each other, Speech detection

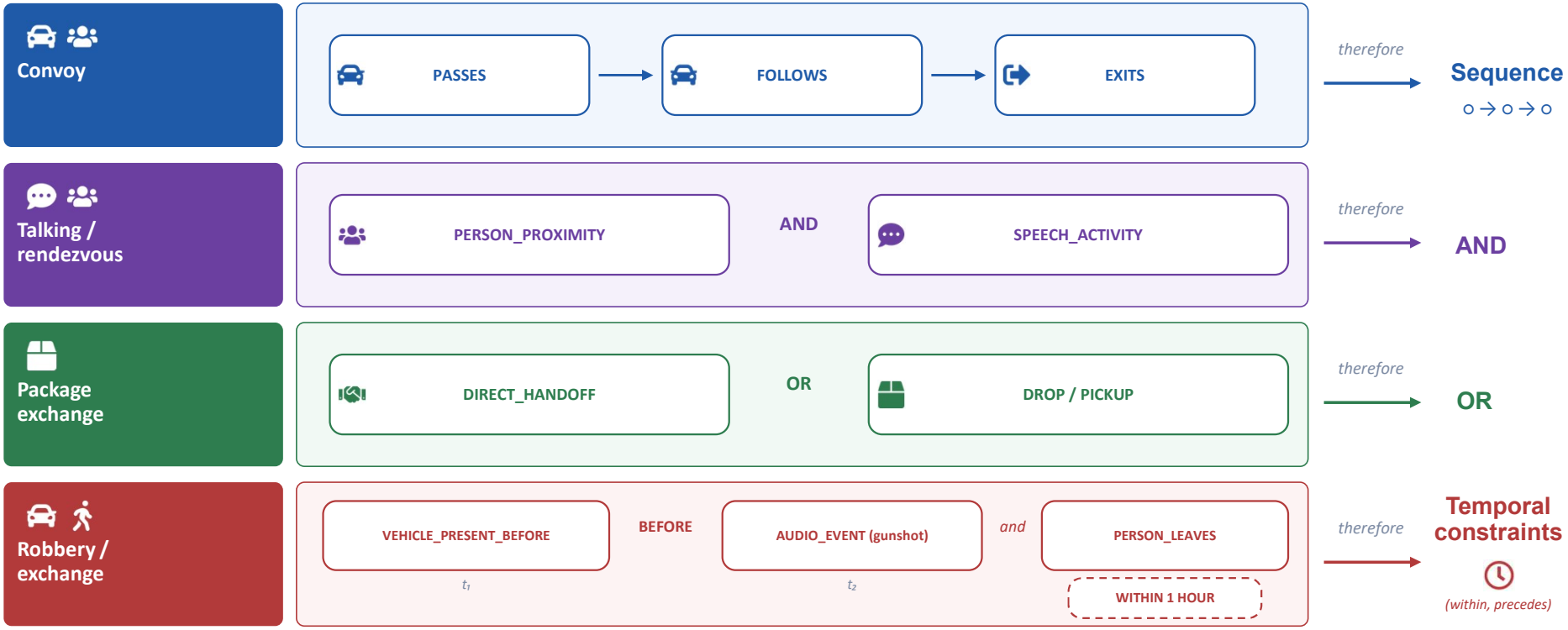
Structure: multiple conditions together

FABLE implements 7 complex events evaluated on real world experiments.

FABLE vocabulary examples

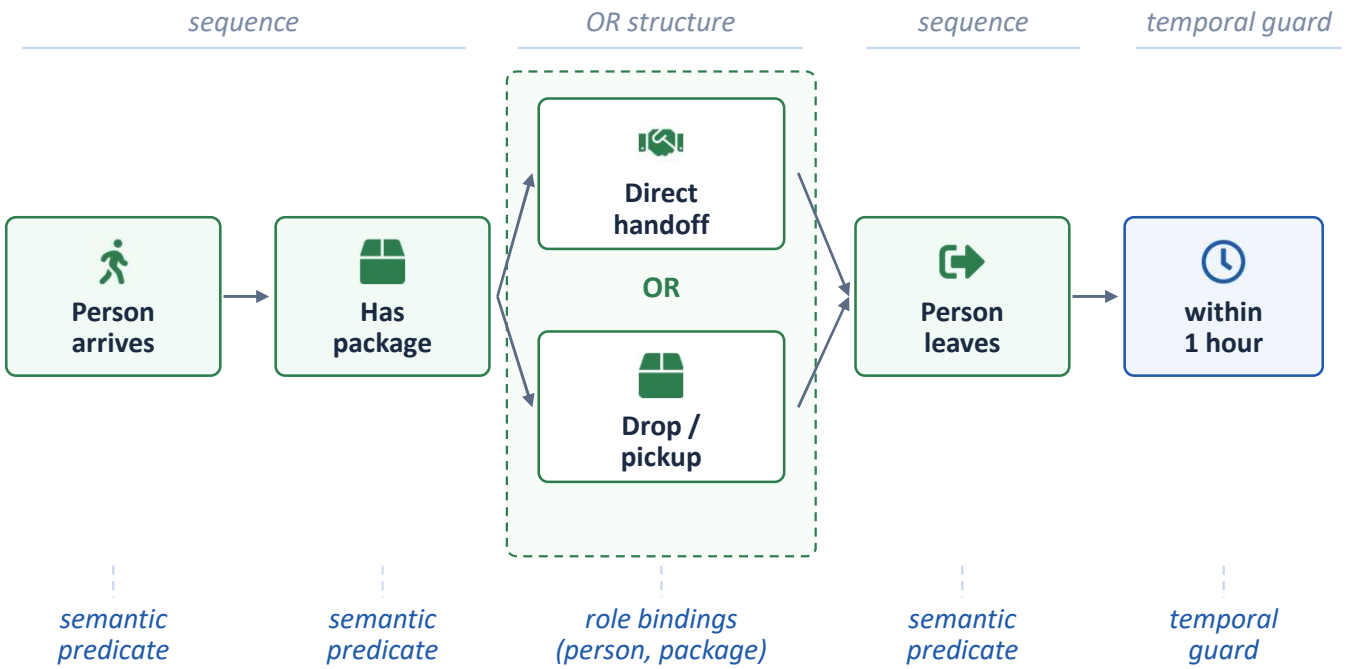
Observed semantic concept	FABLE representation	Example scenario
People, vehicles, packages	→ **Entities**	robbery / convoy / exchange
An entity crosses a reference	→ **PASSES**	convoy
One entity follows another	→ **FOLLOWS**	convoy / chase
An object changes custody	→ **TRANSFER**	package exchange
A sound occurs	→ **AUDIO_EVENT**	robbery
People remain near each other	→ **PERSON_PROXIMITY**	rendezvous
Earlier presence becomes relevant	→ **VEHICLE_PRESENT_BEFORE**	robbery
Entering / leaving a region	→ **ENTERS / EXITS**	convoy / departure

Complex event structure



Fable Event Graph: Example

Example: Package exchange



Semantic predicates

+

Role bindings

+

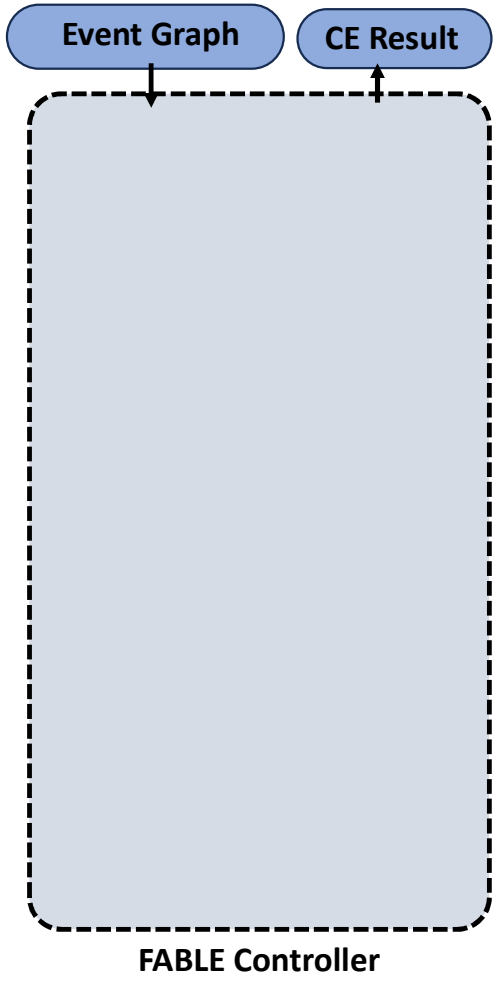
AND/OR/sequence structure

+

Temporal guards

=

FABLE event graph



Questions:

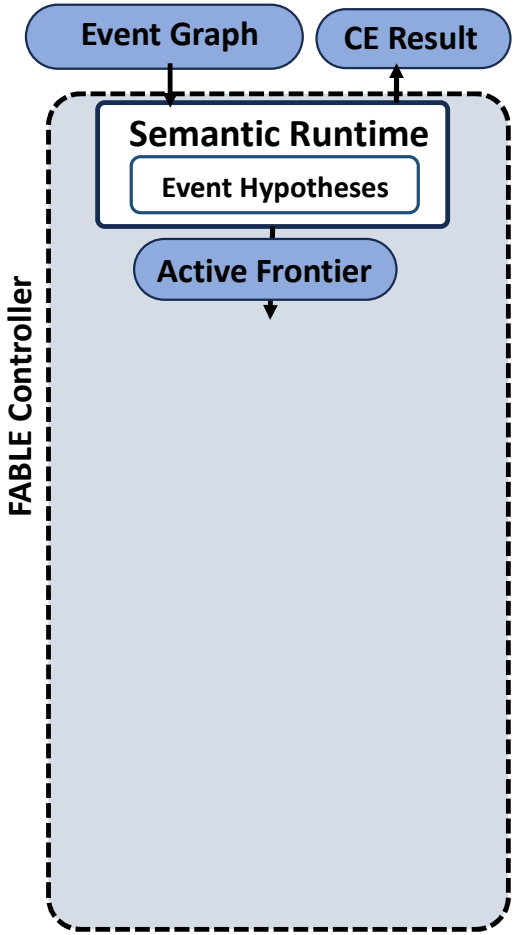
1. **What is the CE representation?**

Implementation: fable/semantic/definitions

Example CE definition

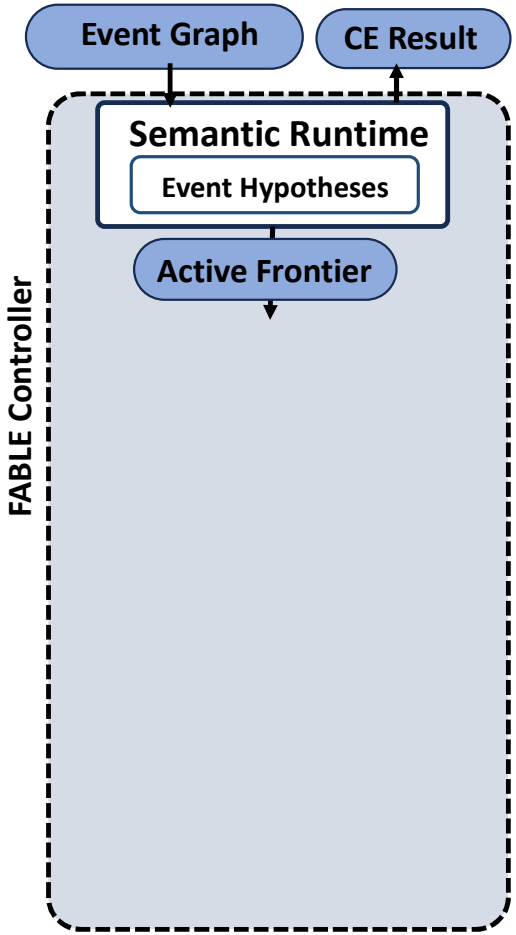
```yaml
semantic_graph:
  event: package_exchange

  roles:  # Declares event-level identity variables that can be shared across multiple predicates.
    vehicle_a: vehicle  # Field = role variable; value = this role must bind to an entity of type "vehicle".
    vehicle_b: vehicle
    package: package  # Event variable representing the package/object being transferred.
    source_holder: person_a  # Person who initially holds the package.
    destination_holder: person_b

  structure:
    sequence:
      - all_of:  # First stage: all listed predicates must become true; their internal order is not important.
          - predicate: ENTERS  # Semantic predicate ID: detect that an entity enters the relevant scene/zone.
            bind:
              vehicle: vehicle_a
          - predicate: ENTERS  # A second ENTERS predicate evaluated independently for the other vehicle.
            bind:
              vehicle: vehicle_b  # This ENTERS predicate applies to the event role "vehicle_b".

      - predicate: TRANSFER  # Second stage: establish that custody/ownership of the package changes.
        bind:
          object: package  # TRANSFER argument "object" is the event's "package".
          source: source_holder  # "source" is the entity giving up the package.
          destination: destination_holder  # "destination" is the entity receiving the package.

      - predicate: EXITS  # Final stage: establish that the receiving vehicle exits/leaves.
        bind:
          vehicle: vehicle_b  # EXITS is specifically evaluated for the previously bound "vehicle_b".
```

Note – many of the examples I give are simplified, the acutal code may reference different objects and have additional parameters



Questions:

1. **What is the CE representation?**
2. **Given the CE representation, what do we need to monitor for?**



Questions:

1. **What is the CE representation?**
2. **Given the CE representation, what do we need to monitor for?**

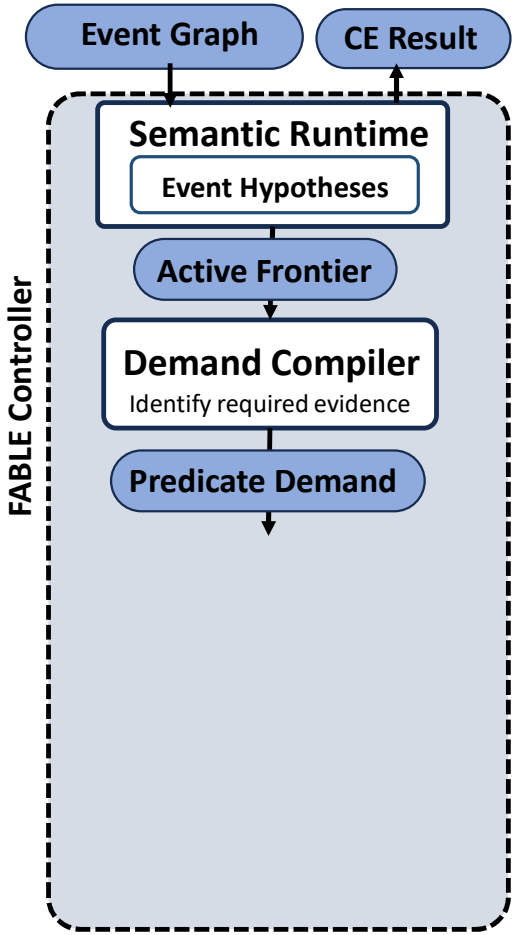
Implementation: fable/semantic/runtime.py

See start() and apply()

Example Hypothesis and Active Frontier

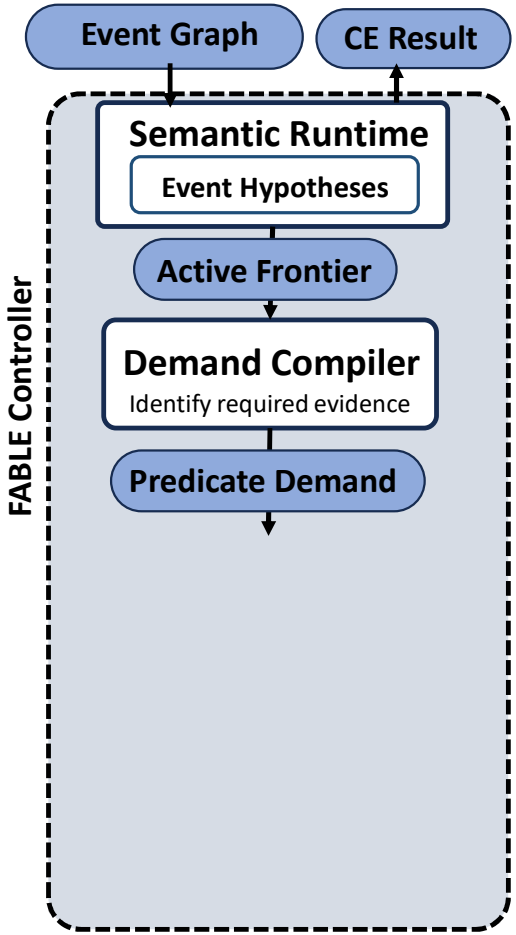
```yaml
hypothesis:
  event: package_exchange  # Event family
  status: ACTIVE  # Lifecycle state, could be ACTIVE, EXPIRED, or COMPLETED

  bindings:  # Canonical identities currently assigned to the graph's event-level role variables.
    vehicle_a: vehicle-A  # Role "vehicle_a" has been resolved to canonical entity "vehicle-A".
    vehicle_b: vehicle-B
    package: package-1  # The package role currently refers to canonical object "package-1".
    source_holder: person-A  # The giver/source role has been resolved to "person-A".
    destination_holder: person-B

  progress:  # Human-readable summary of semantic node state for this particular occurrence.
    arrive_a: satisfied  # The arrival predicate for vehicle_a has already been proven true.
    arrive_b: satisfied  # The arrival predicate for vehicle_b has already been proven true.
    transfer: satisfied  # The TRANSFER predicate has already been proven true.
    receiver_departs: waiting  # The final departure predicate has not yet been satisfied.
```

```yaml
ActiveFrontier:
  active_predicates:
    - arrive_a
    - arrive_b

  checkpoint:
    kind: AND_COMPLETION
    on_success:
      activate: transfer
```



Questions:

1. **What is the CE representation?**
2. **Given the CE representation, what do we need to monitor for?**
3. **What additional constraints are there for how monitoring is done?**



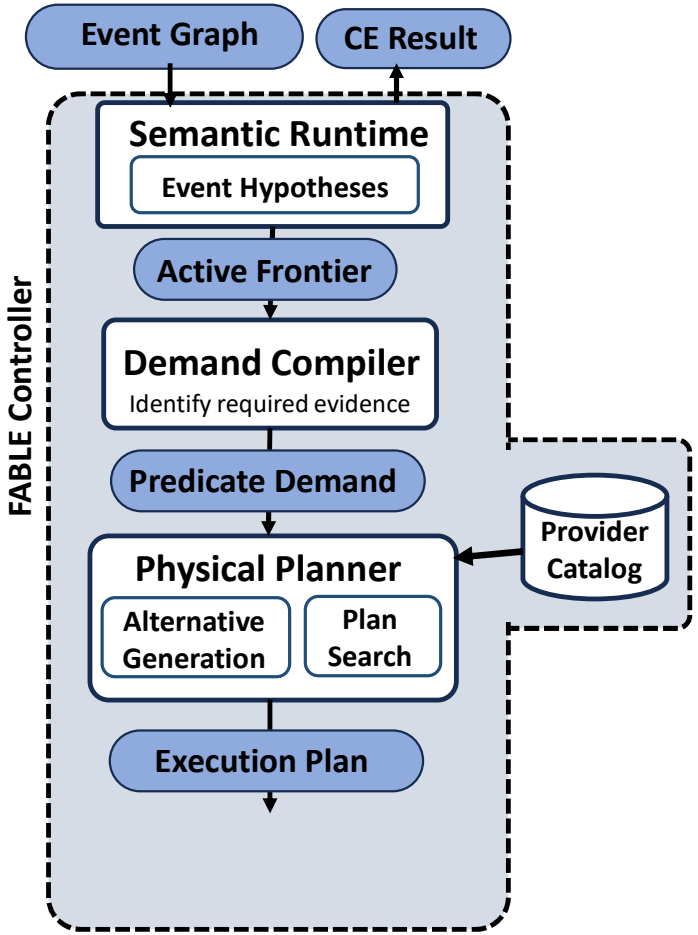
Questions:

1. **What is the CE representation?**
2. **Given the CE representation, what do we need to monitor for?**
3. **What additional constraints are there for how monitoring is done?**

Implementation: fable/planning/demand_compiler.py
See compile()

Example PredicateDemand

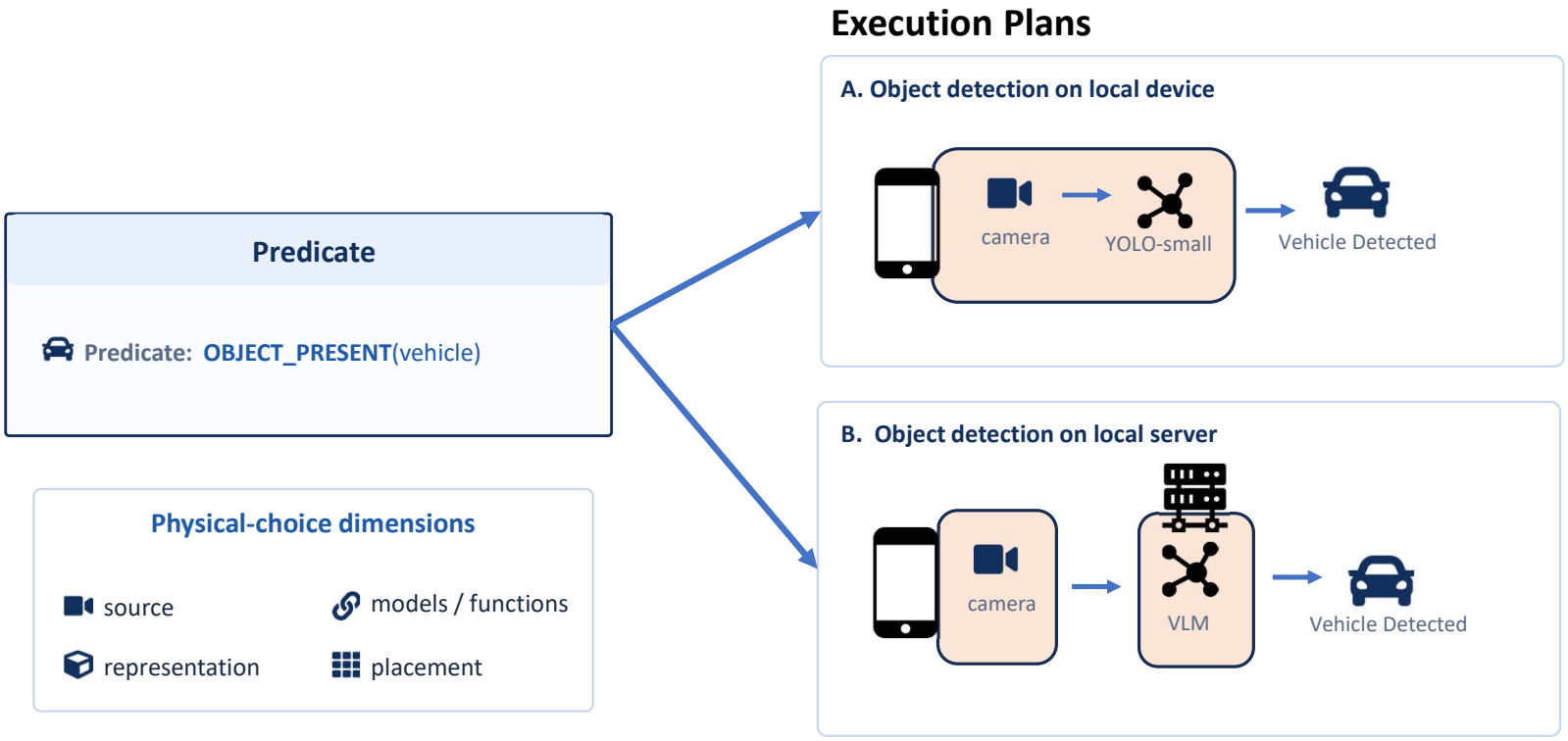
```yaml
predicate_demand:
  predicate: FOLLOWS  # Semantic predicate that must be established for the current frontier.

  roles:
    leader:
      value: vehicle-1  # Already-known canonical entity assigned to "leader".
      mode: VALIDATE  # Provider result must confirm/use this existing identity; it should not replace it.

    follower:
      value: null  # No canonical identity is known yet for this role.
      mode: INTRODUCE  # A valid result is allowed to introduce a new canonical identity for this role.
      may_fork_hypothesis: true  # Different valid follower identities may create separate semantic hypotheses.

  required_capabilities:  # Abstract capabilities needed to satisfy this semantic request.
    - vehicle_identity  # The physical solution must preserve/establish vehicle identity.
    - relative_motion  # The physical solution must reason about relative motion/trajectory.

  acceptable_output:  # Data/result types that can satisfy the semantic boundary.
    - predicate_match.v1  # Canonical predicate-match result type expected by downstream semantic handling.

  event_time_window:  # Time interval over which evidence should be evaluated.
    start: "18:19:52"
    end: "18:20:07"

  deadline:  # Scheduling/planning time constraint.
    latest_useful_completion: "18:24:51"  # A result arriving after this time is no longer useful to this hypothesis.

  eligible_sources:  # Sensor/source IDs that are currently eligible to provide evidence.
    - orin11_replay
    - rpi_replay

  constraints:
    raw_data_must_remain_local: true  # Raw sensor data cannot be moved off its allowed/local sensing node.
```

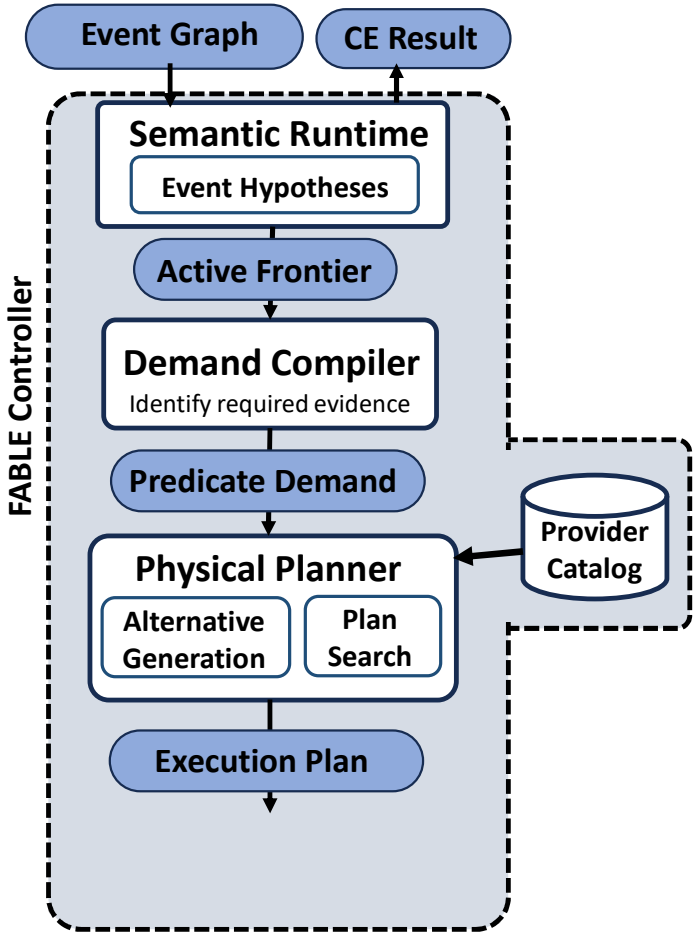


Questions:

1. **What is the CE representation?**
2. **Given the CE representation, what do we need to monitor for?**
3. **What additional constraints are there for how monitoring is done?**
4. **What is the concrete implementation for performing that monitoring?**

One Predicate, Multiple Execution Plans





Questions:

1. **What is the CE representation?**
2. **Given the CE representation, what do we need to monitor for?**
3. **What additional constraints are there for how monitoring is done?**
4. **What is the concrete implementation for performing that monitoring?**

Implementation: fable/providers/registry/catalog.yaml, fable/planning/planner.py

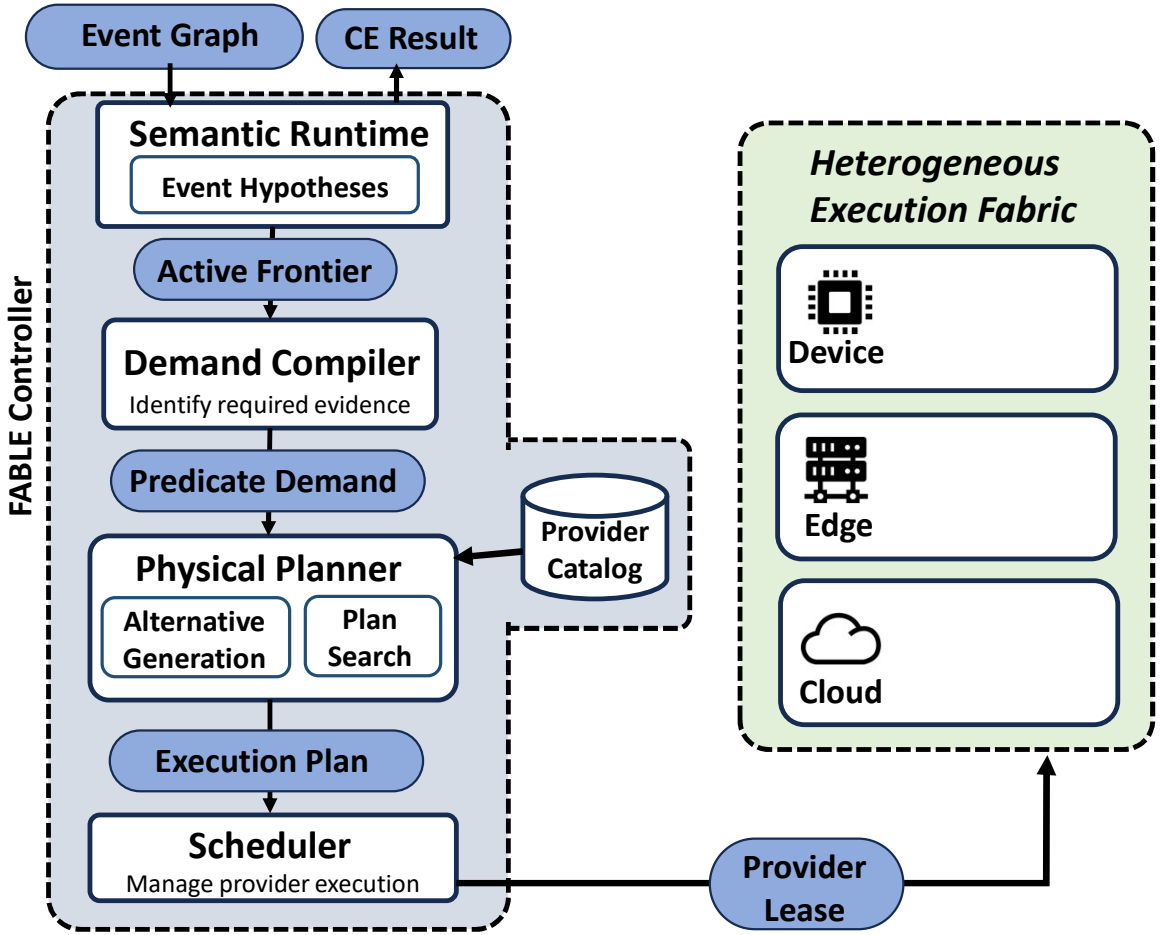
See plan()

Example Provider

```yaml
providers:
  yolo_vehicle_fast_640:  # Stable provider ID
    supports:  # High-level capabilities/operations this provider can contribute.
      - vehicle_detection

    inputs:  # Named input ports exposed by the provider contract.
      frames:  # Port name used when wiring upstream data into the provider.
        type: raw_video_frames.v1  # Typed data contract expected on the "frames" input.
        accepted_input_access:
          - LOCAL  # In this example, raw frames must be available on the same logical execution node.

    outputs:  # Named output ports produced by the provider.
      detections:
        type: detection_set.v1  # Typed contract emitted by the detector.

    resource_profile:  # Simplified resource requirements used during physical planning.
      gpu_required: true
```

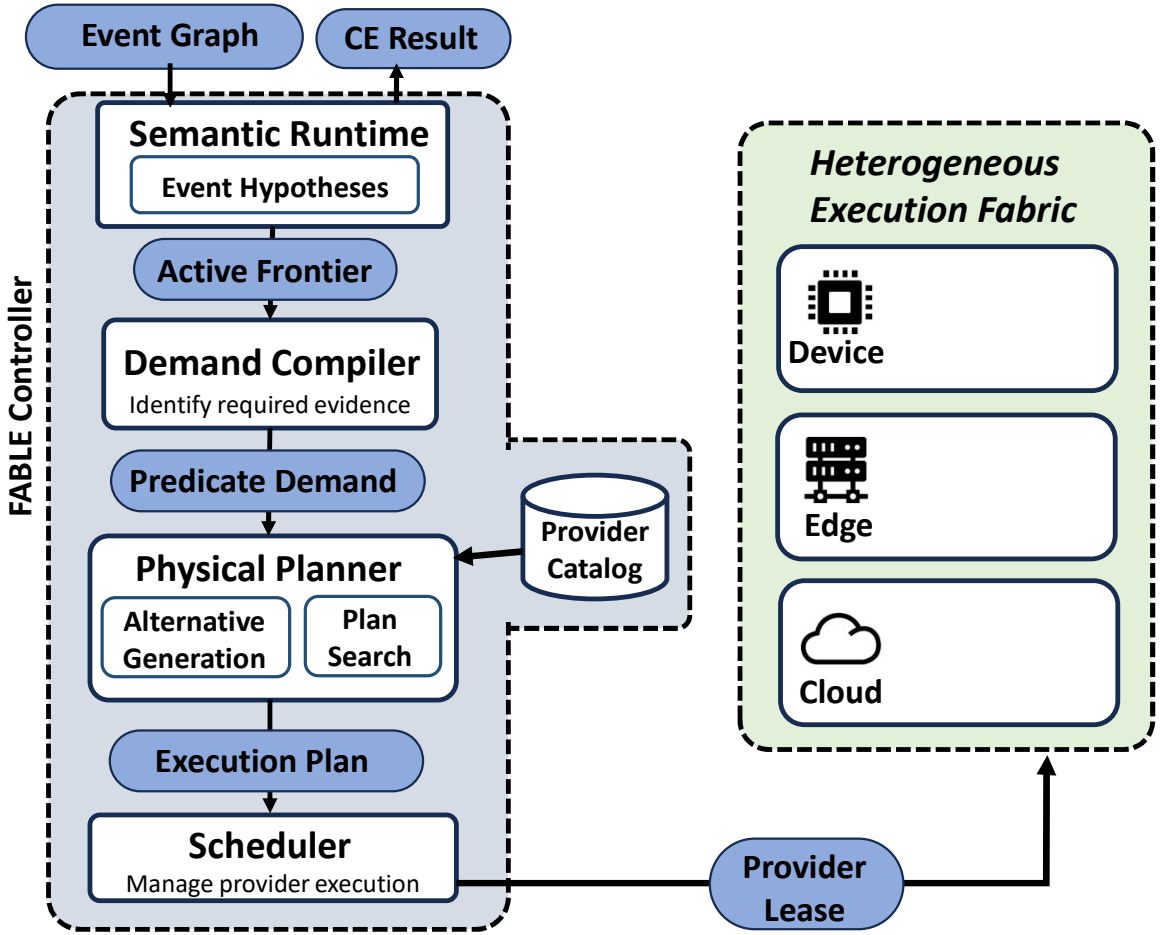
Example ExecutionPlan

```yaml
execution_plan:
  demand: "FOLLOWS(vehicle-1, follower)"  # Human-readable summary

  steps:
    - id: detect  # Stable step name within this plan.
      provider: yolo_vehicle_fast_640
      node: jetson1

    - id: track  # Second plan step.
      provider: multi_object_tracker
      node: jetson1
      after:
        - detect  # Dependency: do not run/use this step until "detect" has produced its required output.

    - id: follows  # Final step that computes the semantic FOLLOWS relation.
      provider: follows_local_geometry  # Concrete implementation chosen for FOLLOWS evaluation.
      node: jetson1  # Final relation check also runs on the Jetson.
      after:
        - track  # Requires projected tracks from the prior step.

  predicted_completion_ms: 6533  # Planner's estimated end-to-end completion time for this realization.
  transfer_bytes: 0  # Estimated inter-node data transfer; zero because all steps are colocated here.
```



Questions:

1. **What is the CE representation?**
2. **Given the CE representation, what do we need to monitor for?**
3. **What additional constraints are there for how monitoring is done?**
4. **What is the concrete implementation for performing that monitoring?**
5. **How is the concrete implementation distributed across devices?**



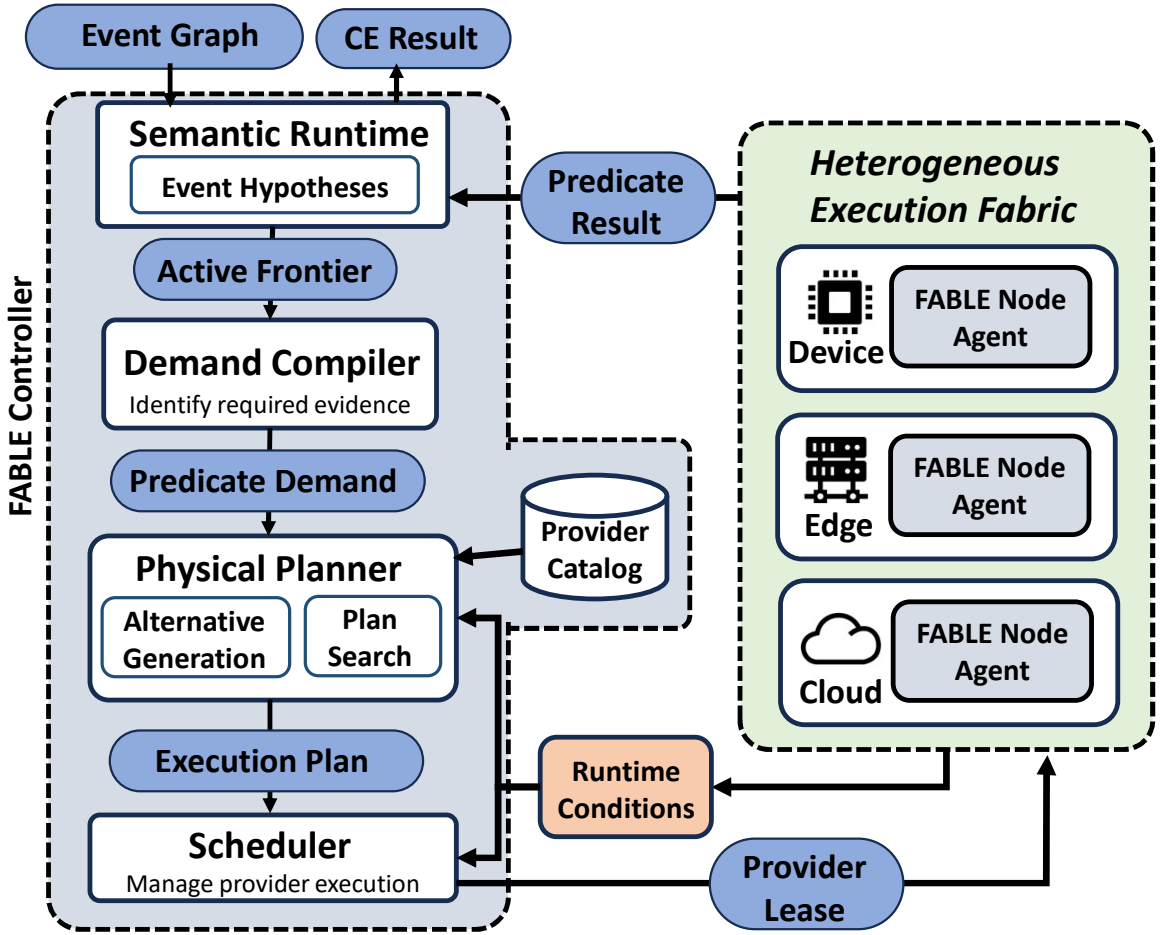
Questions:

1. **What is the CE representation?**
2. **Given the CE representation, what do we need to monitor for?**
3. **What additional constraints are there for how monitoring is done?**
4. **What is the concrete implementation for performing that monitoring?**
5. **How is the concrete implementation distributed across devices?**

Implementation: fable/scheduling/admission.py
See admit()

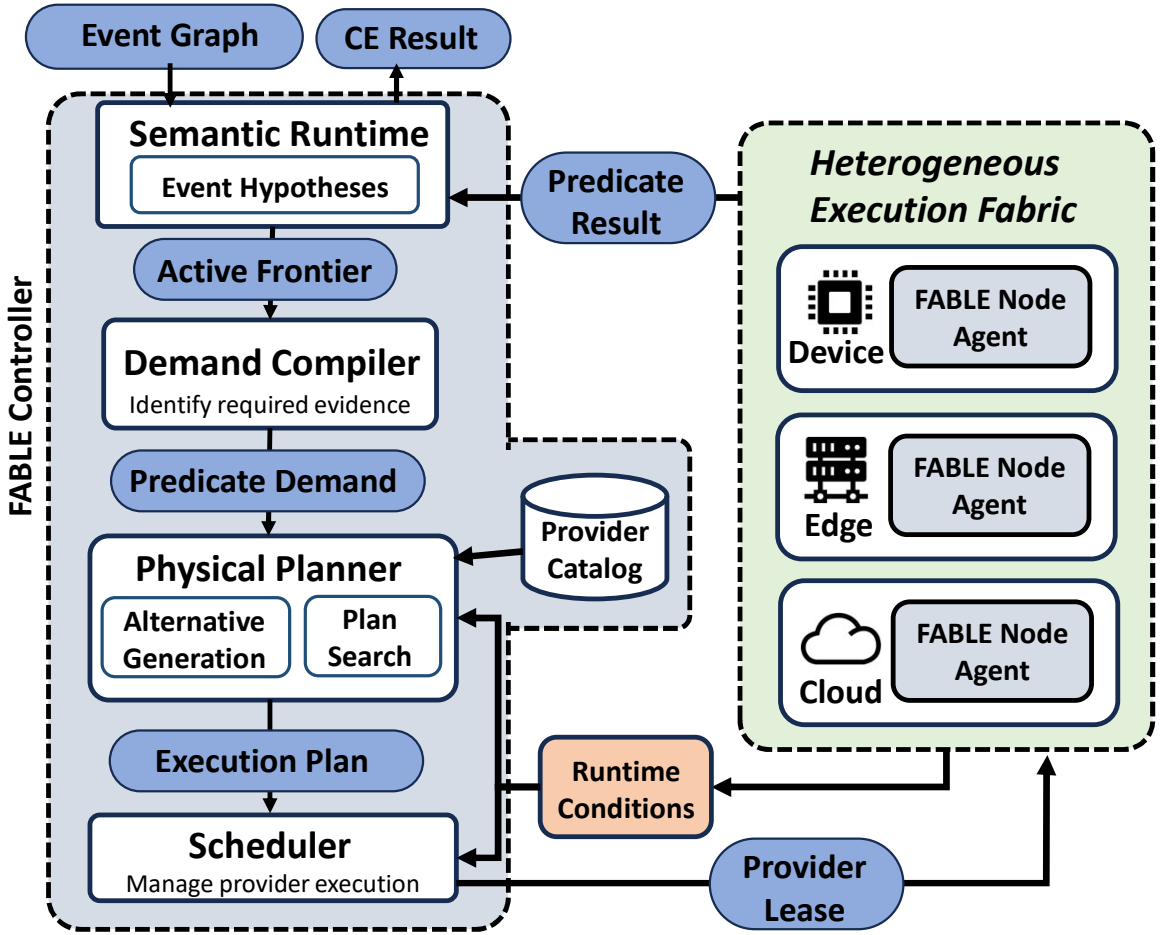
Example ProviderLease

```yaml
provider_lease:
  provider_id: audio_event_classifier  # Stable logical provider implementation being leased.

  provider_instance_id: audio-instance-1  # Identifier of the specific running/warm provider instance.
  node_id: orin11  # physical node where the provider runs

  demand_id: audio-event-demand  # PredicateDemand whose work is authorized by this lease.
  plan_id: audio-event-plan  # ExecutionPlan from which this lease originated.

  status: ACTIVE  # Current lease lifecycle state; ACTIVE means the demand may currently use this instance.

  starts_at: "22:30:34"  # Time from which this lease is valid/attached.
  expires_at: "22:35:34"  # Time after which this lease should no longer be considered valid.
```



Questions:

1. **What is the CE representation?**
2. **Given the CE representation, what do we need to monitor for?**
3. **What additional constraints are there for how monitoring is done?**
4. **What is the concrete implementation for performing that monitoring?**
5. **How is the concrete implementation distributed across devices?**



Questions:

1. **What is the CE representation?**
2. **Given the CE representation, what do we need to monitor for?**
3. **What additional constraints are there for how monitoring is done?**
4. **What is the concrete implementation for performing that monitoring?**
5. **How is the concrete implementation distributed across devices?**

Implementation: fable/distributed/node_agent.py

See activate(), forward_result()

Example: Runtime Conditions

```yaml
runtime_conditions:
  nodes:

    orin11:  # Physical node ID
      available: true  # Whether this node is currently usable for new/existing work.

      free_capacity:  # Current available resources, typically derived from heartbeat/runtime updates.
        cpu_cores: 6.0
        memory_mb: 50000
        gpu_memory_mb: 12000

      heartbeat:  # Freshness/status information reported by the NodeAgent.
        status: HEALTHY
        sequence: 1  # Increasing heartbeat sequence number used to order updates.
        age_ms: 250  # Example age of the most recently received heartbeat.

      provider_runtime:  # Dynamic state associated with provider execution on this node.
        ready_instances: 1  # Number of already-running/warm provider instances available for reuse.

  links:  # Current network conditions between logical deployment nodes.

    - source: orin11  # Starting node of this directed/effective network path.
      target: desktop  # Destination node of the path.

      available: true  # Whether the link/path can currently carry traffic.
      latency_ms: 25  # Current estimated one-way/path latency used by the planner.
      bandwidth_mbps: 50  # Current available/effective bandwidth used for transfer estimates.

  sources:  # Current availability of sensor/replay sources attached to deployment nodes.

    physical_rpi_camera_replay:  # physical source ID
      node: physical_rpi  # Node on which the source is physically/logically located.
      available: true
      data_type: raw_video_frames.v1  # Live data type produced by this source.
```

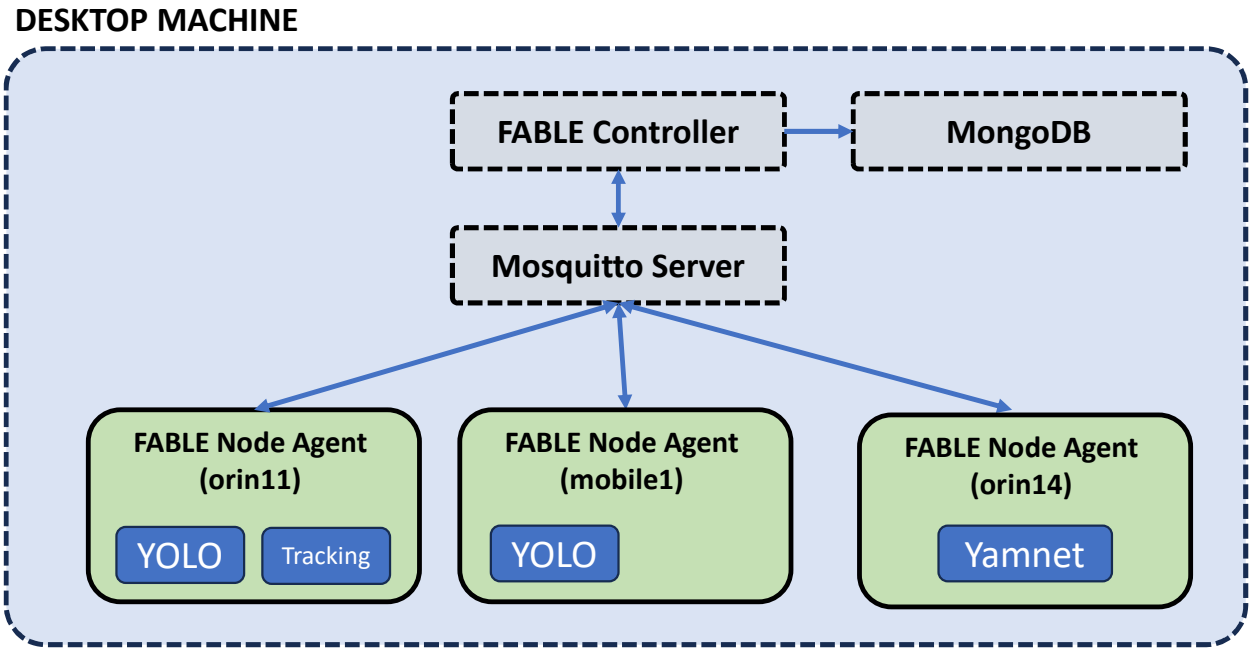
Example: PredicateResult

```yaml
predicate_result:
  predicate: OBJECT_PRESENT  # Semantic predicate this evidence is intended to satisfy.
  truth: true
  confidence: 0.263  # Provider/model confidence associated with this positive result.

  bindings_introduced:  # New semantic role identities learned from this result.
    object: detected-car-1  # The previously unbound "object" role is proposed as canonical entity "detected-car-1".

  event_time:  # Event-time interval to which this evidence applies.
    start: "20:26:58.533"
    end: "20:26:58.533"

  provenance:  # Records where/how the evidence was generated.
    provider_id: yolo_vehicle_fast_640
    node_id: orin11  # Execution node where inference occurred.
    source_id: orin11  # Sensor/replay source whose data was processed.
    model_id: yolov8n
```

FABLE structure

- fable/
  - Catalog/
    - List of predicates used
  - Contracts/
    - Data models used in the architecture (e.g. event graphs, active frontiers, etc)
  - Distributed/
    - Node agents, MQTT, MongoDB
  - Planning/
    - Mainly for planning module
  - Semantic/
    - Complex event definitions, and semantic runtime
  - Scheduling/
    - Mainly for scheduling module
- Providers/
  - Stores information and implementation of providers

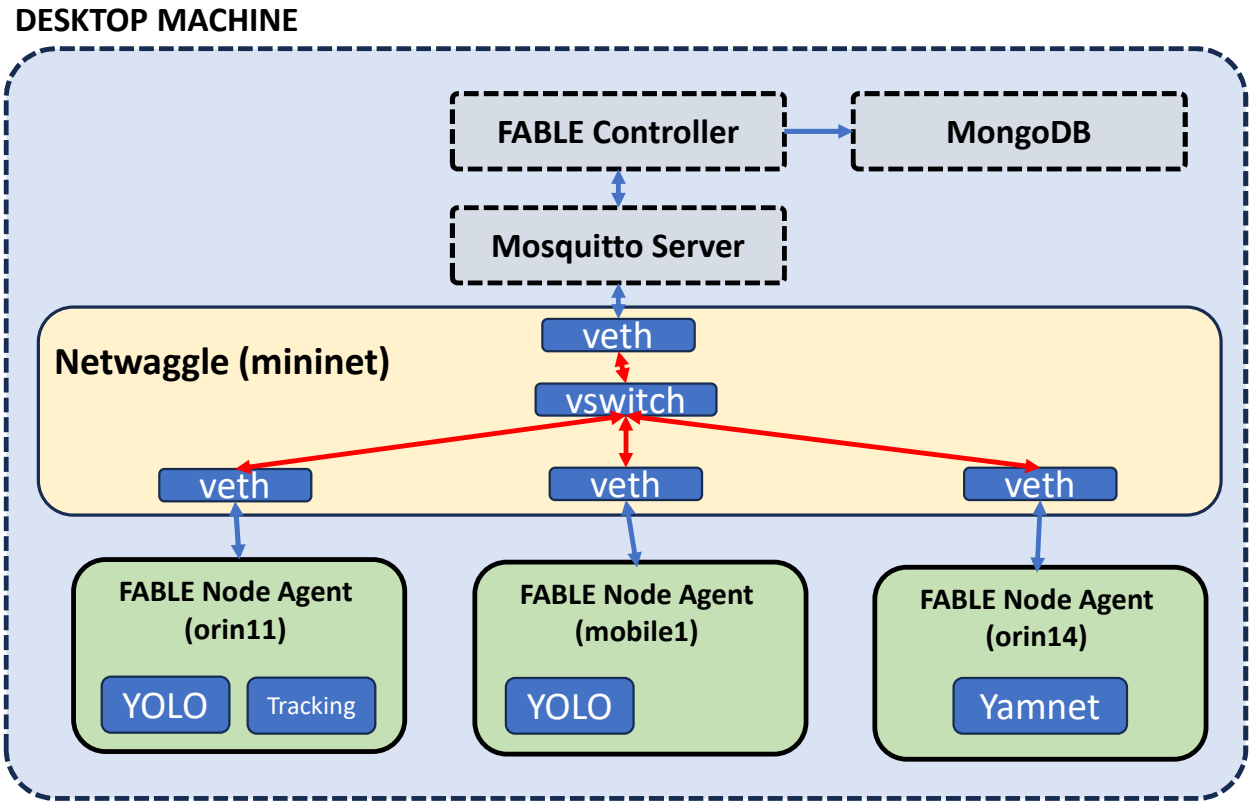
Evaluation Architecture (basic)



Each logical host has a collection of Docker containers

Replay handled by iobt-minimal-ce-replay/services/replay

Evaluation Architecture (with emulated network)



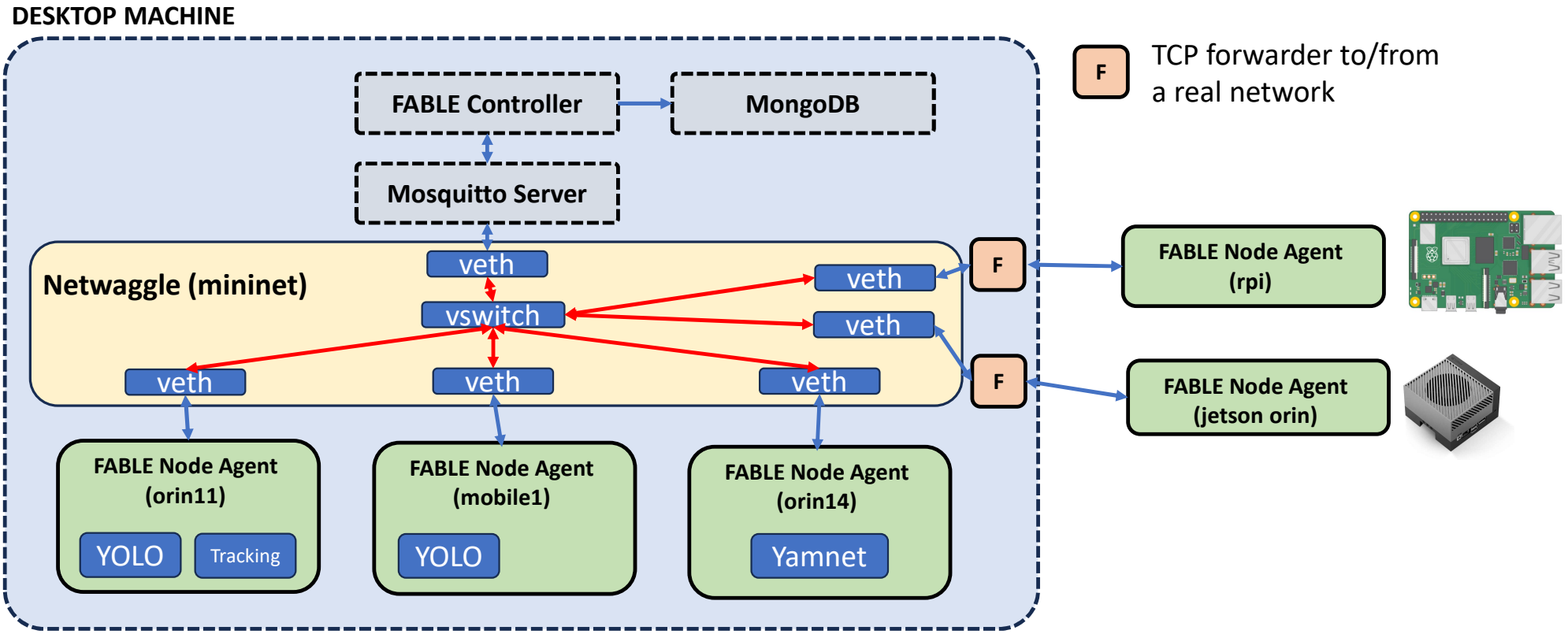
Emulated network links, can change bandwidth, latency, etc

Vswitch – virtual switch in mininet, veth – virtual ethernet interface

Network emulation: netwaggle/netwaggle/runner.py

Network topology configuration: netwaggle/configs

Evaluation Architecture (with real devices)



Vswitch – virtual switch in mininet, veth – virtual ethernet interface